decided that the `std::memory_order_acquire` in `increase_head_count()` was enough, so `std::memory_order_relaxed` is enough there also.

What about other threads? Do you need anything stronger here to ensure other threads are still safe? The answer is, no, because `head` is only ever modified by compare/ exchange operations. Because these are read-modify-write operations, they form part of the release sequence headed by the compare/exchange in `push()`. Therefore, the `compare_exchange_weak()` in `push()` synchronizes with a call to `compare_exchange_strong()` in `increase_head_count()`, which reads the value stored, even if many other threads modify `head` in the meantime.

So you've nearly finished: the only remaining operations to deal with are the `fetch_add()` operations for modifying the reference count. The thread that got to return the data from this node can proceed, safe in the knowledge that no other thread can have modified the node data. However, any thread that did *not* successfully retrieve the data knows that another thread *did* modify the node data; it used `swap()` to extract the referenced data item. Therefore you need to ensure that the `swap()` happens-before the `delete` in order to avoid a data race. The easy way to do this is to make the `fetch_add()` in the successful-return branch use `std::memory_order_release` and the `fetch_add()` in the loop-again branch use `std::memory_order_acquire`. However, this is still overkill: only one thread does the `delete` (the one that sets the count to zero), so only that thread needs to do an acquire operation. Thankfully, because `fetch_add()` is a read-modify-write operation, it forms part of the release sequence, so you can do that with an additional `load()`. If the loop-again branch decreases the reference count to zero, it can reload the reference count with `std::memory_order_acquire` in order to ensure the required synchronizes-with relationship, and the `fetch_add()` itself can use `std::memory_order_relaxed`. The final stack implementation with the new version of `pop()` is shown here.

Listing 7.12. A lock-free stack with reference counting and relaxed atomic operations